

DESIGN PATTERNS AND PRINCIPLES

Exercise 1: Singleton Pattern

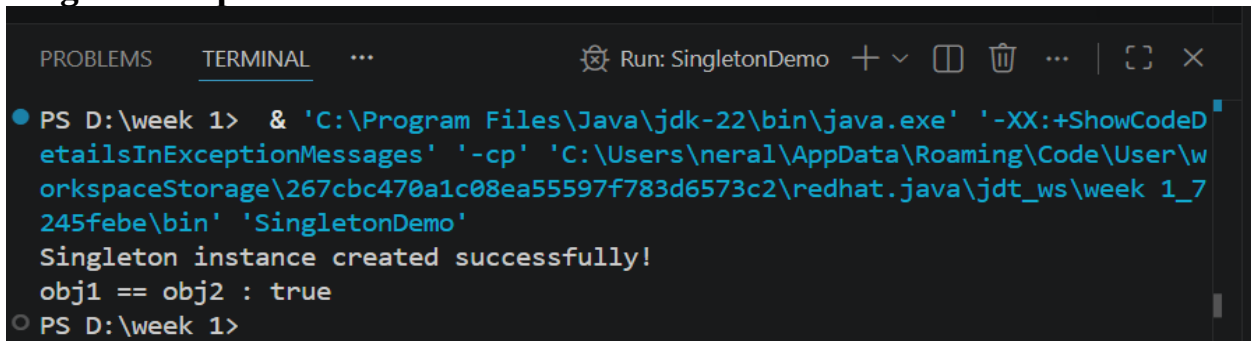
Objective:

To implement the Singleton Design Pattern ensuring only one instance of a class is created.

Code:

```
public class SingletonDemo {
    private static SingletonDemo instance;
    private SingletonDemo() { }
    public static SingletonDemo getInstance() {
        if (instance == null)
            instance = new SingletonDemo();
        return instance;
    }
    public void showMessage() {
        System.out.println("Singleton instance created successfully!");
    }
    public static void main(String[] args) {
        SingletonDemo obj1 = SingletonDemo.getInstance();
        SingletonDemo obj2 = SingletonDemo.getInstance();
        obj1.showMessage();
        System.out.println("obj1 == obj2 : " + (obj1 == obj2));
    }
}
```

Program Output Screenshot:



```
PS D:\week 1> & 'C:\Program Files\Java\jdk-22\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\neral\AppData\Roaming\Code\User\workspaceStorage\267cbc470a1c08ea55597f783d6573c2\redhat.java\jdt_ws\week_1_7245febe\bin' 'SingletonDemo'
Singleton instance created successfully!
obj1 == obj2 : true
PS D:\week 1>
```

Output Text:

```
Singleton instance created successfully!
obj1 == obj2 : true
```

Result:

Only one instance was created and reused.

Conclusion:

Singleton ensures controlled access to a single object.

Exercise 2: Factory Method Pattern

Objective:

To implement the Factory Method Pattern for object creation without exposing instantiation logic.

Code:

```
interface Shape {
    void draw();
}
class Circle implements Shape {
    public void draw() { System.out.println("Drawing Circle"); }
}
class Square implements Shape {
    public void draw() { System.out.println("Drawing Square"); }
}
class ShapeFactory {
    public Shape getShape(String type) {
        if (type.equalsIgnoreCase("CIRCLE")) return new Circle();
        else if (type.equalsIgnoreCase("SQUARE")) return new Square();
        return null;
    }
}
public class FactoryDemo {
    public static void main(String[] args) {
        ShapeFactory factory = new ShapeFactory();
        Shape s1 = factory.getShape("CIRCLE");
        Shape s2 = factory.getShape("SQUARE");
        s1.draw();
        s2.draw();
    }
}
```

Program Output Screenshot:

```
PS D:\week 1> cd "DESIGN PRINCIPLES & PATTERNS"
>>
PS D:\week 1\DESIGN PRINCIPLES & PATTERNS> javac FactoryDemo.java
PS D:\week 1\DESIGN PRINCIPLES & PATTERNS> java FactoryDemo
Drawing Circle
Drawing Square
```

Output Text:

Drawing Circle
Drawing Square

Result:

Objects were created dynamically using the factory method.

Conclusion:

Factory Method simplifies object creation and improves flexibility.